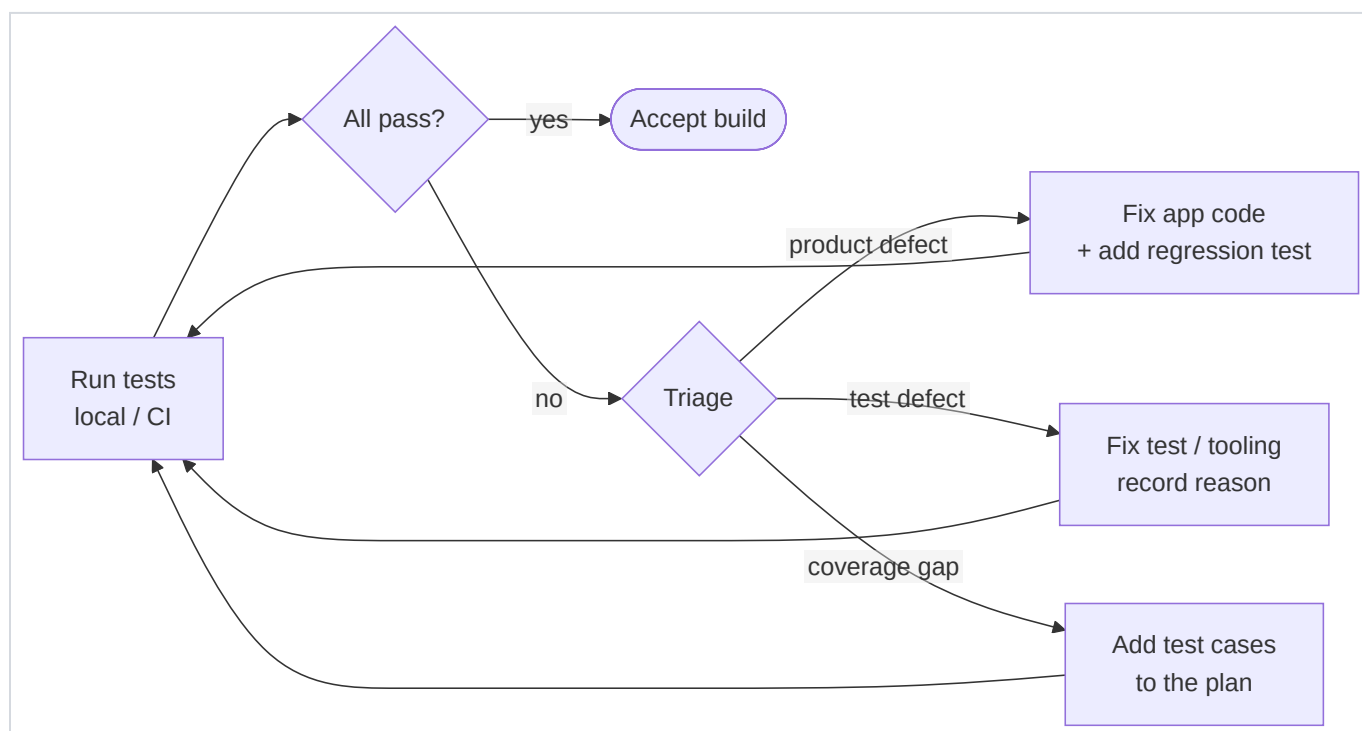


Denarii Dolor — Summary of Changes Resulting from Testing

Covers	The unit test cycle for the validation test plan (2026-09-21), plus earlier test-driven fixes recorded in <code>history.md</code>
Sources	JUnit / JaCoCo reports, GitHub Actions CI logs, <code>git history</code> , <code>history.md</code>

1. How test findings become changes

Every failing test is triaged before anything is changed. A **product defect** is fixed in the app, and the test is kept (or a new one is added) so the defect cannot return. A **test defect** (wrong expectation, fragile setup or tooling problem) is fixed in the test code, and the reason is recorded. Either way, the full suite is re-run and the change is logged in `history.md`.



2. This test cycle: validation unit (TC-01 - TC-20)

2.1 Changes made while planning (coverage gap analysis)

Before the tests were run, each existing test was mapped onto the decision flow of `ValidateTransactionUseCase` (test plan §3.1). The existing 6 tests covered only the reference checks, the transfer rules and the budget rules. **Six of the eleven return failure(...) branches had no test:** invalid amount, blank description, invalid date, invalid account ID,

invalid category ID, and missing transfer destination. There were also no tests for these positive and robustness paths:

- a valid transfer;
- income in a budgeted category;
- a category with no budget;
- a repository error.

Change	Files	Why
9 new test methods (TC-01 - TC-04, TC-06, TC-09, TC-13 - TC-15) and a <code>messageFor()</code> helper	<code>app/src/test/.../ValidateTransactionUseCaseTest.kt</code>	Close the gaps above. The class grew from 6 to 15 tests, and the unit suite from 107 to 116.
A throwing <code>CategoryRepository</code> built with Kotlin interface delegation (by <code>FakeCategoryRepository(...)</code>) inside TC-15	same file	Simulates a database failure without changing the shared fakes

Figure 1. The test cases added from the plan.

```

97  @Test
98  fun rejectsZeroOrNegativeAmount() = runBlocking<Unit> {
99      assertEquals("Invalid amount", messageFor(TestData.expense(amountCents = 0)))
100     assertEquals("Invalid amount", messageFor(TestData.expense(amountCents = -1)))
101 }
102
103 @Test
104 fun rejectsBlankDescription() = runBlocking<Unit> {
105     assertEquals("Description cannot be blank", messageFor(TestData.expense().copy(description = " ")))
106 }
107
108 @Test
109 fun rejectsNonPositiveDate() = runBlocking<Unit> {
110     assertEquals("Invalid date", messageFor(TestData.expense().copy(dateEpochMillis = 0)))
111 }
112
113 @Test
114 fun rejectsNonPositiveAccountOrCategoryIds() = runBlocking<Unit> {
115     assertEquals("Invalid account", messageFor(TestData.expense(accountId = 0)))
116     assertEquals("Invalid category", messageFor(TestData.expense(categoryId = 0)))
117 }
118
119 @Test
120 fun transferRequiresDestination() = runBlocking<Unit> {
121     val transfer = TestData.expense().copy(type = TransactionType.TRANSFER, categoryId = 3, transferAccountId = null)
122
123     assertEquals("Transfer destination is required", messageFor(transfer))
124 }
125
126 @Test
127 fun validTransferPasses() = runBlocking<Unit> {
128     val transfer = TestData.expense().copy(type = TransactionType.TRANSFER, categoryId = 3, accountId = 1, transferAccountId = 2)
129
130     assertTrue(useCase(transfer).isSuccess)
131 }
132
133 @Test
134 fun budgetOnlyLimitsExpenses() = runBlocking<Unit> {
135     transactions.expenseTotalCents = 9_000
136     val income = TestData.expense(amountCents = 5_000).copy(type = TransactionType.INCOME)
137
138     assertTrue(useCase(income).isSuccess)
139 }
140
141 @Test
142 fun expenseWithoutBudgetIsNotLimited() = runBlocking<Unit> {
143     transactions.expenseTotalCents = 1_000_000
144
145     assertTrue(useCase(TestData.expense(amountCents = 500_000, categoryId = 1)).isSuccess)
146 }
147
148 @Test
149 fun repositoryErrorBecomesFailureInsteadOfCrash() = runBlocking<Unit> {
150     val failingCategories = object : CategoryRepository by FakeCategoryRepository(TestData.categories) {
151         override suspend fun getById(id: Long): CategoryEntity? = throw IllegalStateException("database closed")
152     }
153     val failingUseCase = ValidateTransactionUseCase(failingCategories, FakeAccountRepository(TestData.accounts), budgets, transactions)
154
155     val result = failingUseCase(TestData.expense())
156
157     assertFalse(result.isSuccess)
158     assertEquals("database closed", result.exceptionOrNull()?.message)
159 }
160
161 private suspend fun messageFor(transaction: TransactionEntity): String? = useCase(transaction).exceptionOrNull()?.message
162 }

```

Before being run, the new code was checked with ktlint 1.3.1 and detekt 1.23.8 (the CI versions):
0 findings.

2.2 Changes resulting from the test run

Result	Change
20 / 20 plan cases passed; 116 / 116 unit tests passed; 0 failures and 0 errors	No production code changed. The validation unit behaved as specified on the first run.
ValidateTransactionUseCase coverage: 97.4 % of lines, 94.4 % of branches	No change. The three partially covered lines were analysed (results §3): one compiler artifact, one coroutine-resumption branch, and one unreachable defensive guard. They were accepted rather than covered with artificial tests.
Line 56's <code>transferAccountId != null</code> guard can't be reached with a null value	Kept on purpose. It protects <code>checkReferencesAndBudget</code> if it is ever called without going through <code>invoke</code> . It is recorded here so a future refactor can remove it deliberately.

3. Earlier changes caused by test failures

These changes came from earlier test runs (local runs and the CI instrumented suite on the API 26 and API 35 emulators).

#	Found by	Finding	Type	Change made	Regression protection
1	CrudScreensTest.dashboardShowsBudgetAlertAndMeterStatus (CI, API 26)	The budget meter showed " 94 % used " for \$95 of \$100, because <code>(0.95f * 100).toInt()</code> truncates to 94 in floating point	Product defect	New <code>BudgetProgress.percentUsed</code> , computed in integer math (<code>spentCents * 100 / limitCents</code>) and used by the meter label	New unit test <code>DashboardMappersTest.percentUsedUsesIntegerMath</code>
2	EncryptedDatabaseTest → initializationError (CI)	<code>@Test fun x() = runBlocking { ... }</code> returned the last expression (<code>AppDatabase</code>), so JUnit rejected the whole class because a test method wasn't void	Test defect	All <code>30 = runBlocking { ... }</code> bodies (unit and instrumented) changed to <code>runBlocking<Unit></code>	The rule is recorded in the maintenance guide

#	Found by	Finding	Type	Change made	Regression protection
3	Same CI run: the dashboard meter wasn't found	The meter was below the fold of a <code>LazyColumn</code> , so it wasn't composed	Test defect + testability	The dashboard list got the test tag <code>DASHBOARD_LIST_TAG</code> ; tests call <code>performScrollToNode</code> before asserting	Tests updated
4	<code>ComposeScreenTest.loginScreenPrimaryActionLabelMatchesMode</code>	"Cannot call <code>setContent</code> twice"	Test defect	One <code>setContent</code> , driven by <code>mutableStateOf(mode)</code>	—
5	<code>ComposeScreenTest.loginScreenWipeDialogDismissRequestTriggersCancelCallback</code> (two iterations)	A back press never reached the dialog window. Espresso <code>pressBack()</code> then failed with <code>RootViewWithoutFocusException</code>	Test defect	<code>UiDevice.pressBack()</code> from <code>UiAutomator</code> (dependency <code>androidx.test.ui.automator:uiautomator:2.3.0</code> added)	Confirmed by the next CI instrumented run
6	Manual test of Forgot PIN	"Back to Sign In" did nothing. <code>switchToSignIn()</code> kept the screen in recovery mode, and the system back gesture closed the app	Product defect	<code>refreshLoginState(preserveRecoveryMode = false)</code> , plus a <code>BackHandler</code> in recovery mode	New instrumented test <code>recoveryModeBackButtonAndSystemBackReturnToSignIn</code> (41 instrumented tests)
7	CI <code>lintDebug</code>	<code>ui-test-junit4</code> : resolved with no version	Build defect	The Compose BOM was added to <code>androidTestImplementation</code> , and <code>local.properties</code> was untracked	CI green
8	CI <code>ktlint</code> / <code>detekt</code> (blocking)	70+ style findings and 41 static-analysis findings	Code quality	<code>ktlint --format</code> , constants renamed to <code>SCREAMING_SNAKE</code> , complex methods split (e.g. <code>LoginActivity.wipe flow</code>), magic numbers named	Both tools blocking in CI; 0 findings
9	Independent review of test code before first compile	<code>import androidx.compose.ui.test.assertDoesNotExist</code> (a member function, so it can't be imported)	Test defect	Import removed in <code>ComposeScreenTest</code> and <code>CrudScreenTest</code>	Compiles in CI

#	Found by	Finding	Type	Change made	Regression protection
10	Code review (Phase 5); locked in by new tests LedgerTest.centsAvoidFloatingPointDrift and MoneyTest.centsSumExactly	Double amounts drift, e.g. $0.1 + 0.2 \neq 0.3$	Design defect	All money moved to Long cents; database migration v1 → v2 (MIGRATION_1_2)	MoneyTest, LedgerTest, instrumented MigrationTest
11	Fragile test doubles	Anonymous per-test repository fakes broke whenever a repository interface grew	Test maintainability	Shared testutil/FakeRepositories.kt and a TestData fixture	Used by every use-case test, including this plan

Figure 2. Finding #1, a product defect found by a failing test. The fix and the unit test that locks it in.

```

git diff — percent fix

diff --git a/.../presentation/ui/dashboard/DashboardMappers.kt b/app/src/main/java/com/denariidolor/presentation/ui/dashboard/DashboardMappers.kt
index 226ea93..e46031f 100644
--- a/app/src/main/java/com/denariidolor/presentation/ui/dashboard/DashboardMappers.kt
+++ b/app/src/main/java/com/denariidolor/presentation/ui/dashboard/DashboardMappers.kt
@@ -43,6 +43,9 @@ data class BudgetProgress(
    val warningPercent: Int
  ) {
    val fraction: Float get() = if (limitCents > 0) (spentCents.toDouble() / limitCents).toFloat() else 0f
+
+  /** Whole percent, computed in integer math so 95% never renders as 94% from float error. */
+  val percentUsed: Int get() = if (limitCents > 0) (spentCents * PERCENT / limitCents).toInt() else 0
    val status: BudgetStatus get() = budgetStatus(spentCents, limitCents, warningPercent)
  }

diff --git a/.../presentation/ui/dashboard/DashboardScreen.kt b/app/src/main/java/com/denariidolor/presentation/ui/dashboard/DashboardScreen.kt
@@ -238,7 +241,7 @@ private fun BudgetMeter(progress: BudgetProgress) {
    Icon(imageVector = icon, contentDescription = null, tint = color, modifier = Modifier.size(16.dp))
    Spacer(modifier = Modifier.width(4.dp))
    Text(
-      text = stringResource(R.string.budget_status_line, statusLabel, (progress.fraction * 100).toInt()),
+      text = stringResource(R.string.budget_status_line, statusLabel, progress.percentUsed),
      style = MaterialTheme.typography.bodySmall,
      color = MaterialTheme.colorScheme.onSurfaceVariant
    )
@@ -180,4 +181,11 @@ class DashboardMappersTest {
    assertEquals("$1.5K", compactMoney(1_500f))
    assertEquals("$2.0M", compactMoney(2_000_000f))
  }
+
+  @Test
+  fun percentUsedUsesIntegerMath() {
+    // 0.95f * 100 truncates to 94 in floating point; the displayed value must be 95.
+    assertEquals(95, BudgetProgress(1, "Dining", "dining", spentCents = 9_500, limitCents = 10_000, warningPercent = 80).percentUsed)
+    assertEquals(0, BudgetProgress(1, "Dining", "dining", spentCents = 500, limitCents = 0, warningPercent = 80).percentUsed)
+  }
}

```

Figure 3. Finding #6, a product defect. The fix to the Forgot PIN "Back to Sign In" button and the system back gesture.

```
git diff — backtosignin fix

@@ -87,6 +88,8 @@ fun LoginScreen(
    onCancelWipeData: () -> Unit,
    onConfirmWipeData: () -> Unit
) {
+    BackHandler(enabled = mode == LoginScreenMode.RECOVER_PIN && signInEnabled, onBack = onBackToSignIn)
+
    val biometricContentDescription = stringResource(R.string.biometric_sign_in_accessibility_label)
    val wipeActionContentDescription = stringResource(R.string.wipe_all_data_destructive_label)
    val wipeConfirmContentDescription = stringResource(R.string.wipe_data_confirm_destructive_label)
diff --git a/app/src/main/java/com/denariidolor/presentation/ui/auth/LoginActivity.kt b/app/src/main/java/com/denariidolor/presentation/ui/auth/LoginActivity.kt
index 109c486..99582da 100644
--- a/app/src/main/java/com/denariidolor/presentation/ui/auth/LoginActivity.kt
+++ b/app/src/main/java/com/denariidolor/presentation/ui/auth/LoginActivity.kt
@@ -235,7 +235,7 @@ class LoginActivity : AppCompatActivity() {
    return
}
clearInputs()
- refreshLoginState()
+ refreshLoginState(preserveRecoveryMode = false)
feedbackMessage = null
}
```

Figure 4. Finding #2, a test defect. Making `runBlocking` test bodies return `Unit` so JUnit accepts them.

```
git diff — runblocking fix

diff --git a/app/src/androidTest/java/com/denariidolor/data/EncryptedDatabaseTest.kt b/app/src/androidTest/java/com/denariidolor/data/EncryptedDatabaseTest.kt
index e2f5c71..7a47a67 100644
--- a/app/src/androidTest/java/com/denariidolor/data/EncryptedDatabaseTest.kt
+++ b/app/src/androidTest/java/com/denariidolor/data/EncryptedDatabaseTest.kt
@@ -55,7 +55,7 @@ class EncryptedDatabaseTest {
    .build()

    @Test
    - fun databaseFileIsEncryptedAndReadableWithSameKey() = runBlocking {
+ fun databaseFileIsEncryptedAndReadableWithSameKey() = runBlocking<Unit> {
        val key = DatabaseKeys.generateHexKey(SecureRandom())
        open(key).apply {
```

4. Outcome

Measure	Before	After
JVM unit tests	107	116 (all passing)
ValidateTransactionUseCase tests	6	15
Validation failure branches with a test	5 of 11	11 of 11
Instrumented tests (CI, API 26 and 35)	39 (4 failing)	41 (the 4 failures fixed; confirm with the latest CI run)
Product defects fixed as a result of testing	—	2 UI defects (#1, #6) + 1 design defect (#10)

The change in §2.1 is uncommitted in the working tree, alongside `history.md`, for review before committing.